

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL XI
Error Handling



Disusun Oleh:
Erlangga Aditya Permana 105224038

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA

2026

I. Pendahuluan

Java Exception/Error Handling merupakan mekanisme untuk menangani kondisi tidak normal yang muncul saat program dijalankan, seperti input yang tidak sesuai, data yang tidak ditemukan, atau proses yang gagal memenuhi aturan tertentu. Dalam Java, penanganan exception umumnya dilakukan menggunakan blok try, catch, dan finally, sedangkan throw dan throws digunakan untuk melempar serta mendeklarasikan exception. Mekanisme ini membuat alur program utama tetap rapi karena proses normal dapat dipisahkan dari proses penanganan kesalahan. Program yang dibahas pada laporan ini menerapkan konsep tersebut dalam bentuk sistem reservasi tiket kereta api sederhana, dengan fitur melihat jadwal, memesan tiket, memvalidasi data penumpang, memeriksa kode kereta, dan memastikan jumlah kursi masih tersedia.

II. Variable

No	Nama Variabel	Tipe Data	Fungsi
1	input	Scanner	Membaca masukan pengguna dari keyboard pada menu utama.
2	sistem	SistemReserve	Objek utama yang menyimpan jadwal kereta dan memproses reservasi.
3	running	boolean	Mengontrol perulangan menu agar program tetap berjalan sampai pengguna keluar.
4	menu	int	Menyimpan pilihan menu yang dimasukkan pengguna.
5	kode	String	Menyimpan kode kereta yang dimasukkan saat proses pemesanan.
6	nik	String	Menyimpan NIK penumpang yang akan divalidasi.
7	nama	String	Menyimpan nama penumpang yang melakukan pemesanan.
8	jumlah	int	Menyimpan jumlah tiket yang ingin dipesan.
9	daftarKereta	List<KeretaApi>	Menyimpan kumpulan objek KeretaApi di dalam sistem reservasi.
10	kodeKereta	String	Menjadi kode unik untuk membedakan setiap kereta.
11	namaKereta	String	Menyimpan nama kereta, seperti Argo Bromo atau Parahyangan.

12	rutePerjalanan	String	Menyimpan informasi rute perjalanan kereta.
13	sisakursi	int	Menyimpan jumlah kursi yang masih tersedia pada kereta.
14	targetKereta	KeretaApi	Menampung objek kereta yang cocok dengan kode kereta masukan.
15	k	KeretaApi	Variabel iterasi untuk menelusuri setiap data kereta dalam daftarKereta.
16	NIK	String	Parameter pada method pesan untuk mengecek validitas identitas penumpang.
17	namaPenumpang	String	Parameter pada method pesan yang menyimpan nama penumpang.
18	jumlahtiket	int	Parameter pada method pesan yang menyimpan jumlah tiket yang dipesan.
19	pesan	String	Parameter pada constructor exception untuk menyimpan pesan kesalahan.

III. Constructor dan Method

No	Nama Metode	Jenis Metode	Fungsi
1	main	Static method	Main program
2	SistemReserve()	Constructor	Menginisialisasi daftar kereta dan mengisi data awal K01 Argo Bromo dan K02 Parahyangan.
3	pesan(String kodeKereta, String NIK, String namaPenumpang, int jumlahtiket)	Method	Memproses pemesanan tiket dengan validasi NIK, pencarian kode kereta, pengecekan kursi, dan pengurangan sisa kursi jika berhasil.
4	tampilJadwal()	Method	Menampilkan kode kereta, nama kereta, rute perjalanan, dan sisa kursi dari seluruh data kereta.
5	KeretaApi(String kodeKereta, String namaKereta,	Constructor	Mengisi data awal objek kereta api berdasarkan parameter yang diberikan.

	String rute, int sisaKursi)		
6	DataPenumpangTidakValidException (String pesan)	Constructor	Membuat exception khusus ketika NIK penumpang tidak valid.
7	RuteTidakDitemukanException (String pesan)	Constructor	Membuat exception khusus ketika kode kereta atau rute tidak ditemukan.
8	TiketHabisException (String namaKereta, int sisaKursi)	Constructor	Membuat exception khusus ketika tiket yang diminta melebihi sisa kursi dan menyimpan info sisa kursi.

IV. Dokumentasi dan Pembahasan Code

a. Isi Kode

KeretaApi.java :

```
public class KeretaApi {
    String kodeKereta,namaKereta,rutePerjalanan;
    int sisaKursi;

    public KeretaApi(String kodeKereta, String namaKereta, String rute, int
sisaKursi){
        this.kodeKereta = kodeKereta;
        this.namaKereta = namaKereta;
        this.rutePerjalanan = rute;
        this.sisaKursi = sisaKursi;
    }
}
```

SistemReserve.java

```
import java.util.ArrayList;
import java.util.List;

public class SistemReserve{
    List<KeretaApi> daftarKereta = new ArrayList<>();//simpan data berisi
informasi kereta api dalam collection ArrayList

    public SistemReserve() {
        //tersimpan di memori sistem reservasi
    }
}
```

```

        daftarKereta.add(new KeretaApi("K01", "Argo Bromo", "JKT - SBY", 50));
        daftarKereta.add(new KeretaApi("K02", "Parahyangan", "JKT - BDG",
15));
    }

    public void pesan(String kodeKereta, String NIK, String namaPenumpang, int
jumlahtiket) throws RuteTidakDitemukanException, TiketHabisException{
        //1. cek NIK
        //pakai java stream (chars untuk memecah menjadi karakter-karakter,
allMatch untuk mengecek apakah semua karakter itu sesuai dengan kondisi
isDigit)

                                if(NIK.length() != 16 ||
!NIK.chars().allMatch(Character::isDigit)){//kondisi jika tidak terpenuhi
                throw new DataPenumpangTidakValidException("NIK tidak valid!");
            }

        //2. cek kode kereta (cari dulu keretanya)
        KeretaApi targetKereta = null;
        for (KeretaApi k : daftarKereta) {
            if (k.kodeKereta.equalsIgnoreCase(kodeKereta)) {
                targetKereta = k;
                break;
            }
        }
        if (targetKereta == null){//baru cek kode
            throw new RuteTidakDitemukanException("Kode kereta '" + kodeKereta
+ "' atau rute tidak ditemukan!");
        }

        //3. cek sisa kursi
        if (jumlahtiket > targetKereta.sisaKursi) {
            throw new TiketHabisException(targetKereta.namaKereta,
targetKereta.sisaKursi);
        }

        //jika semua kondisi terpenuhi dan pemesanan tiket berhasil
        targetKereta.sisaKursi -= jumlahtiket;
        System.out.println("Reservasi berhasil.");
    }

    public void tampilJadwal(){
        for(KeretaApi k : daftarKereta){

```

```

        System.out.println("\n");
        System.out.println("Kode kereta : " + k.kodeKereta);
        System.out.println("Nama kereta : " + k.namaKereta);
        System.out.println("Rute kereta : " + k.rutePerjalanan);
        System.out.println("Sisa kursi : " + k.sisaKursi);
        System.out.println("");
    }
}
}

```

DataPenumpangTidakValidException.java :

```

public class DataPenumpangTidakValidException extends RuntimeException{
    public DataPenumpangTidakValidException(String pesan){
        super(pesan);
    }
}

```

RuteTidakDitemukanException.java :

```

public class RuteTidakDitemukanException extends Exception{
    public RuteTidakDitemukanException(String pesan){
        super(pesan);
    }
}

```

TiketHabisException.java :

```

public class TiketHabisException extends Exception{
    String namaKereta;
    int sisaKursi;
    public TiketHabisException(String namaKereta, int sisaKursi){
        super("Pemesanan gagal! Tiket untuk kereta " + namaKereta + " tidak mencukupi sisa kursi.");
        this.namaKereta = namaKereta;
        this.sisaKursi = sisaKursi;
    }
}

```

App.java :

```

import java.util.InputMismatchException;

```

```

import java.util.Scanner;

public class App {
    public static void main(String[] args) throws Exception {
        Scanner input = new Scanner(System.in);
        SistemReserve sistem = new SistemReserve();
        boolean running = true;

        while(running){
            try {
                System.out.println("\n\tMENU UTAMA:");
                System.out.println("1. Lihat Jadwal");
                System.out.println("2. Pesan Tiket");
                System.out.println("3. Keluar");
                System.out.print("Pilih menu (1-3): ");
                int menu = input.nextInt();
                input.nextLine();//buffer input

                switch (menu) {
                    case 1:
                        System.out.println("\t\tJadwal Kereta");
                        sistem.tampilJadwal();
                        break;
                    case 2:
                        System.out.print("Masukkan Kode Kereta: ");
                        String kode = input.nextLine();
                        System.out.print("Masukkan NIK Penumpang: ");
                        String nik = input.nextLine();
                        System.out.print("Masukkan Nama Penumpang: ");
                        String nama = input.nextLine();
                        System.out.print("Masukkan Jumlah Tiket: ");
                        int jumlah = input.nextInt();
                        input.nextLine();

                        sistem.pesan(kode, nik, nama, jumlah);
                        break;
                    case 3:
                        running = false;
                        break;
                    default:
                        System.out.println("Pilihan tidak valid.");
                }
            }
        }
    }
}

```

```

        }catch (InputMismatchException e){//exception handler untuk
masukukan bukan angka (jika huruf/simbol)
            System.out.println("Error : Harap masukkan angka, bukan
huruf/simbol!");
            input.nextLine();//lanjut di-input buffer
        }catch(DataPenumpangTidakValidException e){
            System.out.println("Error : "+ e.getMessage());
        }catch (RuteTidakDitemukanException e) {
            System.out.println("Error : " + e.getMessage());
        }catch (TiketHabisException e) {
            System.out.println("Error : " + e.getMessage());
            System.out.println("Info Sisa Tiket " + e.namaKereta + ": " +
e.sisaKursi + " kursi.");
        }finally {
            //saat keluar dari sistem/loop
            if (!running){
                input.close();
            }
        }
    }
}
}
}

```

b. Alur Program

1. Inisialisasi Sistem

Program membuat objek Scanner untuk membaca input dan objek SistemReserve untuk menjalankan proses reservasi. Saat objek SistemReserve dibuat, constructor langsung mengisi daftar kereta awal, yaitu K01 Argo Bromo rute JKT - SBY dengan 50 kursi dan K02 Parahyangan rute JKT - BDG dengan 15 kursi.

2. Menampilkan Menu Utama

Program masuk ke perulangan while selama running bernilai true. Di dalam perulangan, program menampilkan pilihan menu berupa Lihat Jadwal, Pesan Tiket, dan Keluar.

3. Membaca Pilihan Menu

Pengguna memasukkan angka menu 1 sampai 3. Jika pengguna memasukkan huruf atau simbol pada input angka, InputMismatchException akan ditangkap sehingga program dapat menampilkan pesan error tanpa langsung berhenti.

4. Proses Lihat Jadwal

Jika pengguna memilih menu 1, program memanggil method tampilJadwal(). Method ini menelusuri setiap objek KeretaApi di daftarKereta, lalu menampilkan kode kereta, nama kereta, rute perjalanan, dan sisa kursi.

5. Proses Input Pemesanan

Jika pengguna memilih menu 2, program meminta kode kereta, NIK penumpang, nama penumpang, dan jumlah tiket. Data tersebut kemudian dikirim ke method `pesan()` milik objek `SistemReserve`.

6. Validasi Data Penumpang

Di dalam method `pesan()`, NIK diperiksa terlebih dahulu. NIK harus memiliki panjang 16 karakter dan seluruh karakternya harus berupa angka. Jika syarat ini tidak terpenuhi, program melempar `DataPenumpangTidakValidException`.

7. Validasi Kode Kereta atau Rute

Setelah NIK valid, program mencari kereta berdasarkan kode yang dimasukkan pengguna. Pencarian dilakukan dengan menelusuri `daftarKereta` dan membandingkan kode menggunakan `equalsIgnoreCase()`. Jika tidak ada kereta yang cocok, program melempar `RuteTidakDitemukanException`.

8. Validasi Ketersediaan Tiket

Jika kereta ditemukan, program memeriksa jumlah tiket yang diminta. Apabila jumlah tiket lebih besar daripada sisa kursi pada kereta tersebut, program melempar `TiketHabisException` dan menyimpan informasi nama kereta serta sisa kursinya.

9. Penyelesaian Reservasi

Jika semua validasi berhasil, jumlah sisa kursi pada `targetKereta` dikurangi sesuai jumlah tiket yang dipesan. Setelah itu program menampilkan pesan bahwa reservasi berhasil.

10. Penanganan Kesalahan

Setiap exception ditangani pada blok `catch` di method `main()`. `InputMismatchException` menangani input angka yang salah, `DataPenumpangTidakValidException` menangani NIK tidak valid, `RuteTidakDitemukanException` menangani kode kereta yang tidak ada, dan `TiketHabisException` menangani tiket yang tidak mencukupi sekaligus menampilkan sisa kursi.

11. Keluar dari Program

Jika pengguna memilih menu 3, nilai `running` diubah menjadi `false` sehingga perulangan berhenti. Pada blok `finally`, `Scanner` ditutup ketika program benar-benar keluar dari sistem.

5. Output Program

```
MENU UTAMA:
```

```
1. Lihat Jadwal
```

```
2. Pesan Tiket
```

```
3. Keluar
```

```
Pilih menu (1-3): a
```

```
Error : Harap masukkan angka, bukan huruf/symbol!
```

```
MENU UTAMA:
```

```
1. Lihat Jadwal
```

```
2. Pesan Tiket
```

3. Keluar

Pilih menu (1-3): 1

Jadwal Kereta

Kode kereta : K01

Nama kereta : Argo Bromo

Rute kereta : JKT - SBY

Sisa kursi : 50

Kode kereta : K02

Nama kereta : Parahyangan

Rute kereta : JKT - BDG

Sisa kursi : 15

MENU UTAMA:

1. Lihat Jadwal

2. Pesan Tiket

3. Keluar

Pilih menu (1-3): 2

Masukkan Kode Kereta: K01

Masukkan NIK Penumpang: 1234567890123456

Masukkan Nama Penumpang: Budi

Masukkan Jumlah Tiket: 5

Reservasi berhasil.

MENU UTAMA:

1. Lihat Jadwal

2. Pesan Tiket

3. Keluar

Pilih menu (1-3): 2

Masukkan Kode Kereta: K03

Masukkan NIK Penumpang: 1234567890123457

Masukkan Nama Penumpang: Didi

Masukkan Jumlah Tiket: 2

Error : Kode kereta 'K03' atau rute tidak ditemukan!

MENU UTAMA:

1. Lihat Jadwal

2. Pesan Tiket

3. Keluar

Pilih menu (1-3): 2

Masukkan Kode Kereta: K02

Masukkan NIK Penumpang: 1234567890123457

Masukkan Nama Penumpang: Didi

Masukkan Jumlah Tiket: 100

Error : Pemesanan gagal! Tiket untuk kereta Parahyangan tidak mencukupi sisa kursi.

Info Sisa Tiket Parahyangan: 15 kursi.

MENU UTAMA:

1. Lihat Jadwal

2. Pesan Tiket

3. Keluar

Pilih menu (1-3): 2

Masukkan Kode Kereta: K02

Masukkan NIK Penumpang: 123456789012345

Masukkan Nama Penumpang: Didi

Masukkan Jumlah Tiket: 1

Error : NIK tidak valid!

V. Kesimpulan

Berdasarkan program yang dibuat, konsep exception handling pada Java dapat digunakan untuk membuat proses reservasi lebih terstruktur dan aman terhadap kesalahan input maupun kondisi bisnis yang tidak terpenuhi. Program memisahkan alur normal, seperti menampilkan jadwal dan menyimpan pemesanan, dari alur penanganan error melalui penggunaan try, catch, finally, throw, dan throws. Custom exception seperti DataPenumpangTidakValidException, RuteTidakDitemukanException, dan TiketHabisException juga membantu memperjelas jenis exception yang terjadi. Program tidak hanya berjalan sesuai fungsi utama, tetapi juga mampu memberikan pesan kesalahan yang lebih spesifik dan mudah dipahami oleh pengguna.

VI. Daftar Pustaka

W3schools. Java Exceptions. Diakses pada 8 Juni 2026, dari https://www.w3schools.com/java/java_try_catch.asp

W3schools. Java Multiple Exceptions. Diakses pada 8 Juni 2026, dari https://www.w3schools.com/java/java_exceptions_multiple.asp